Image-Based Retrieval Augmented Generation (RAG) Relevant to Biodiversity and Taxonomy

Talks & Brainstorms – Apr 30, 2024

Nate Lesperance
Department of Integrative Biology

Presentation Objectives

Introduce Retrieval Augmented Generation (RAG) + initial context of RAG bootcamp project

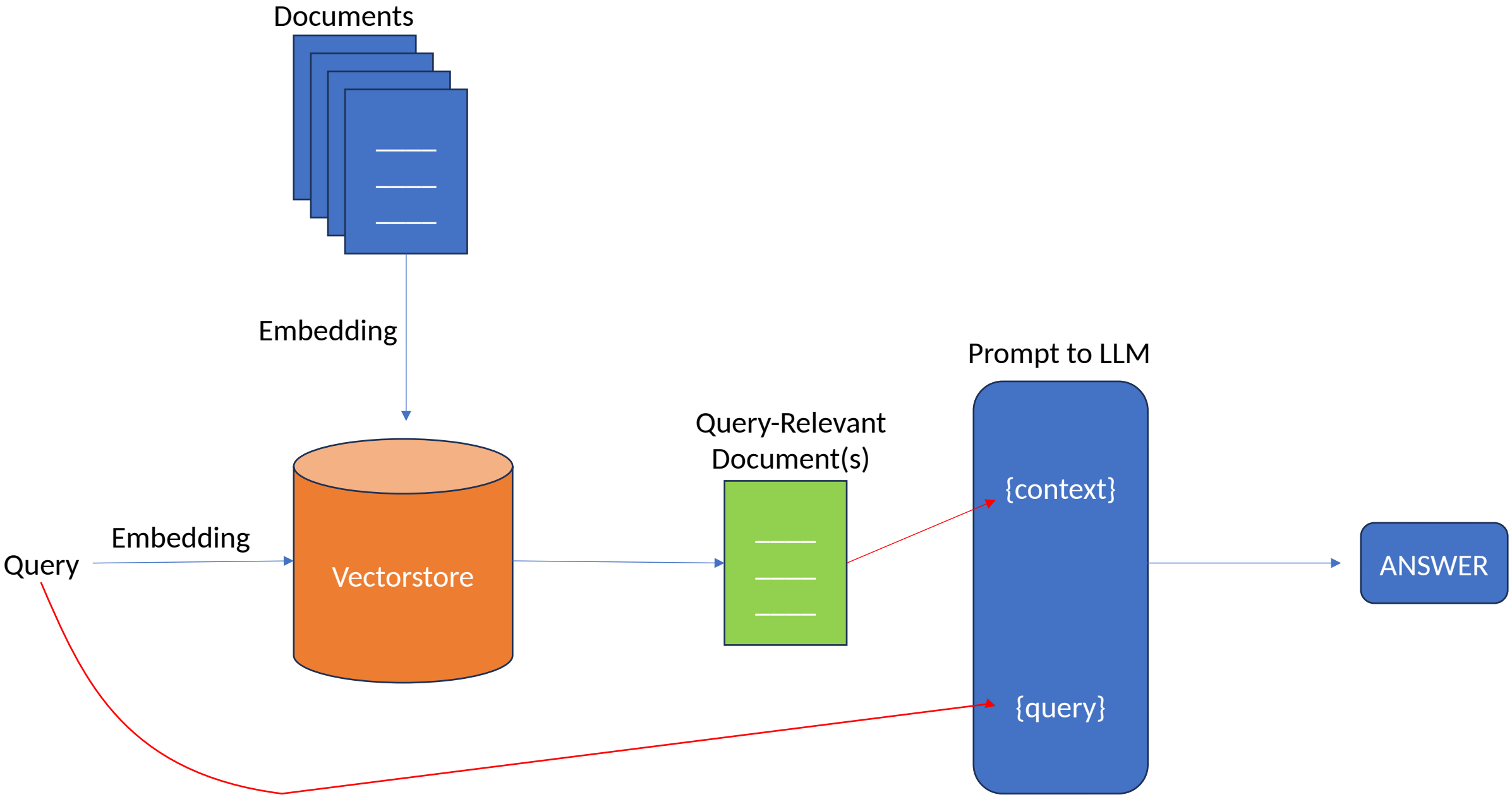
Introduce project objectives using two-module pipeline

Module 1: Framework for image to text for taxonomy and biodiversity

Module 2: Explore RAG optimizations explored so far

Discuss optimizations and next steps moving forward (immediate and distant)

RAG Basics


Documents
Embedding
Vectorstore
Query
Embedding
Query-Relevant
Document(s)
Prompt to LLM
{context}
{query}
ANSWER

Simple RAG Pipeline Example

```python
# Load docs and split into chunks
loader = DirectoryLoader(directory_path, glob="*.txt")
text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=100)
chunks = text_splitter.split_documents(loader.load())
```

```python
# Set up embeddings model and embed doc chunks
embeddings = HuggingFaceBgeEmbeddings(
    model_name=model_name,
    model_kwargs={'device': device},
    encode_kwargs={'normalize_embeddings': True})
vectorstore = Chroma.from_documents(documents=chunks, embedding=embeddings)
retriever = vectorstore.as_retriever()
```

```python
# Set up prompt
prompt = ChatPromptTemplate.from_template("""
Answer the question based only on the following context:
{context}

Question: {question}
""")
```

```python
# RAG Chain
rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser())
```

```python
# Query RAG Chain
print(rag_chain.invoke("""
What wing shape and colouration are found in Heliconiinae butterflies?"""))
```

Full RAG Pipeline (minus some imports and helper functions)

Simple RAG Pipeline Example

```python
# Load docs and split into chunks
loader = DirectoryLoader(directory_path, glob="*.txt")
text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=100)
chunks = text_splitter.split_documents(loader.load())
```

```python
# Set up embeddings model and embed doc chunks
embeddings = HuggingFaceBgeEmbeddings(
    model_name=model_name,
    model_kwargs={'device': device},
    encode_kwargs={'normalize_embeddings': True})
vectorstore = Chroma.from_documents(documents=chunks, embedding=embeddings)
retriever = vectorstore.as_retriever()
```

Load and split documents into chunks

Import Embedding model

Build Vectorstore from embeddings of chunks

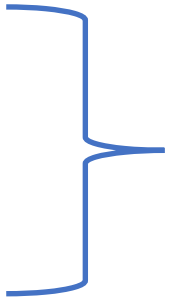
Simple RAG Pipeline Example

```
# Load docs and split into chunks
loader = DirectoryLoader(directory_path, glob="*.txt")
text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=100)
chunks = text_splitter.split_documents(loader.load())
```

```
# Set up embeddings model and embed doc chunks
embeddings = HuggingFaceBgeEmbeddings(
    model_name=model_name,
    model_kwargs={'device': device},
    encode_kwargs={'normalize_embeddings': True})
vectorstore = Chroma.from_documents(documents=chunks, embedding=embeddings)
retriever = vectorstore.as_retriever()
```

```
# Set up prompt
prompt = ChatPromptTemplate.from_template("""
Answer the question based only on the following context:
{context}

Question: {question}
""")
```



Prepare LLM prompt with formattable **context** and **question**

Simple RAG Pipeline Example

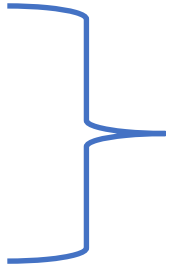
```
# Load docs and split into chunks
loader = DirectoryLoader(directory_path, glob="*.txt")
text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=100)
chunks = text_splitter.split_documents(loader.load())
```

```
# Set up embeddings model and embed doc chunks
embeddings = HuggingFaceBgeEmbeddings(
    model_name=model_name,
    model_kwargs={'device': device},
    encode_kwargs={'normalize_embeddings': True})
vectorstore = Chroma.from_documents(documents=chunks, embedding=embeddings)
retriever = vectorstore.as_retriever()
```

```
# Set up prompt
prompt = ChatPromptTemplate.from_template("""
Answer the question based only on the following context:
{context}

Question: {question}
""")
```

```
# RAG Chain
rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser())
```



Prepare RAG Chain
- Query and context piped to LLM

Simple RAG Pipeline Example

```
# Load docs and split into chunks
loader = DirectoryLoader(directory_path, glob="*.txt")
text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=100)
chunks = text_splitter.split_documents(loader.load())
```

```
# Set up embeddings model and embed doc chunks
embeddings = HuggingFaceBgeEmbeddings(
    model_name=model_name,
    model_kwargs={'device': device},
    encode_kwargs={'normalize_embeddings': True})
vectorstore = Chroma.from_documents(documents=chunks, embedding=embeddings)
retriever = vectorstore.as_retriever()
```

```
# Set up prompt
prompt = ChatPromptTemplate.from_template("""
Answer the question based only on the following context:
{context}

Question: {question}
""")
```

```
# RAG Chain
rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser())
```

```
# Query RAG Chain
print(rag_chain.invoke("""
What wing shape and colouration are found in Heliconiinae butterflies?"""))
```

Query the RAG Chain
- Ask your question

Simple RAG Pipeline Example

```python
# Load docs and split into chunks
loader = DirectoryLoader(directory_path, glob="*.txt")
text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=100)
chunks = text_splitter.split_documents(loader.load())
```

```python
# Set up embeddings model and embed doc chunks
embeddings = HuggingFaceBgeEmbeddings(
    model_name=model_name,
    model_kwargs={'device': device},
    encode_kwargs={'normalize_embeddings': True})
vectorstore = Chroma.from_documents(documents=chunks, embedding=embeddings)
retriever = vectorstore.as_retriever()
```

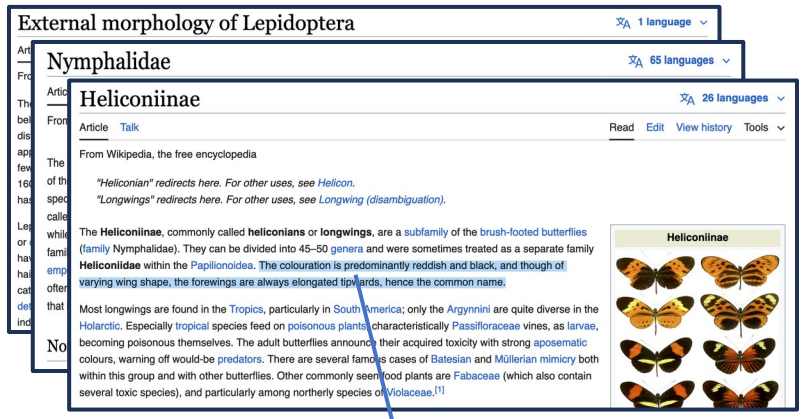
```python
# Set up prompt
prompt = ChatPromptTemplate.from_template("""
Answer the question based only on the following context:
{context}

Question: {question}
""")
```

```python
# RAG Chain
rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser())
```

```python
# Query RAG Chain
print(rag_chain.invoke("""
What wing shape and colouration are found in Heliconiinae butterflies?"""))
```

Vectorstore from few Wikipedia articles



Relevant Context:

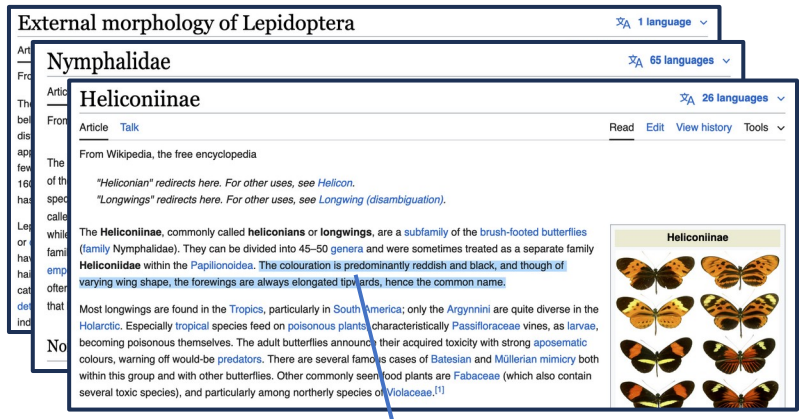
The colouration is predominantly reddish and black, and though of varying wing shape, the forewings are always elongated tipwards, hence the common name.

Query:

What wing shape and colouration are found in Heliconiiae butterflies?

Simple RAG Pipeline Example

Vectorstore from few Wikipedia articles



Result

The answer is Heliconiinae butterflies are predominantly black in colouration and have varying wing shapes, particularly elongated wings towards the tip.

Relevant Context:
The colouration is predominantly reddish and black, and though of varying wing shape, the forewings are always elongated tipwards, hence the common name.

How is RAG made image-based?

Query:
What wing shape and colouration are found in Heliconiiae butterflies?

Refined Use-Case: Taxonomic Classification + Biodiversity Knowledge

Module 1



Image Captioner

Descriptive Caption

Module 2

Tree of Life Web
Wikipedia
Wikispecies
Scientific Journals
Web search?
Biodiversity Heritage Library
Others?

RAG Pipeline

Output:
i) Taxonomic Classification
ii) Biodiversity Knowledge

Refined Use-Case: Taxonomic Classification + Biodiversity Knowledge

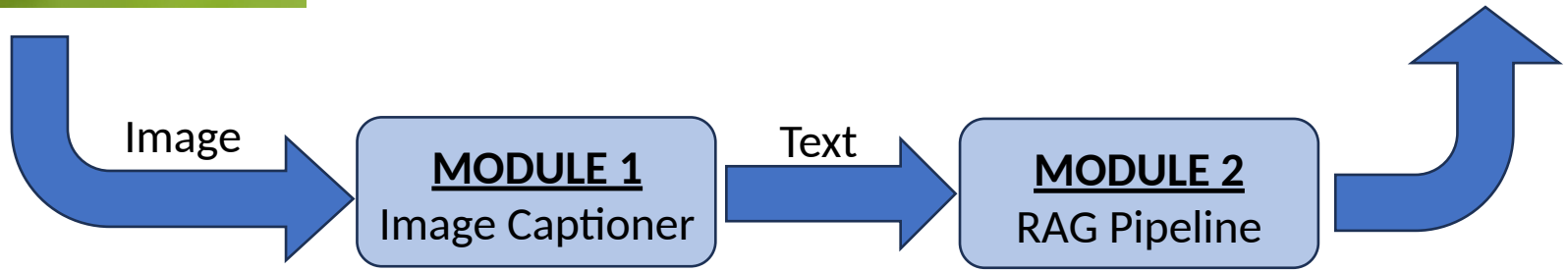
True label: *Euptoieta claudia*



Result
Based on the information provided, the organism you most likely have in the image is a butterfly from the genus **Euptoieta**.

Although I don't have an image to visually confirm this, the description you've provided aligns with the common characteristics of the Euptoieta genus, particularly **Euptoieta hegesia**, also known as the variegated fritillary or the banded fritillary.

Within the context of the information you've provided, here's why Euptoieta is a suitable candidate:
1. Wing color and pattern: Euptoieta butterflies have distinctive brown and black wings with white or yellow bands, which matches the description of the wing color and pattern seen in your image.
2. **Floral preference:** Your description indicates that the butterfly was seen perching on a white flower, which is consistent with the floral preferences of Euptoieta species. They are often found on white or yellow flowers, especially those in the umbellifer family.
3. Range and habitat: Different Euptoieta species are found in various regions, and some are common in suburban areas

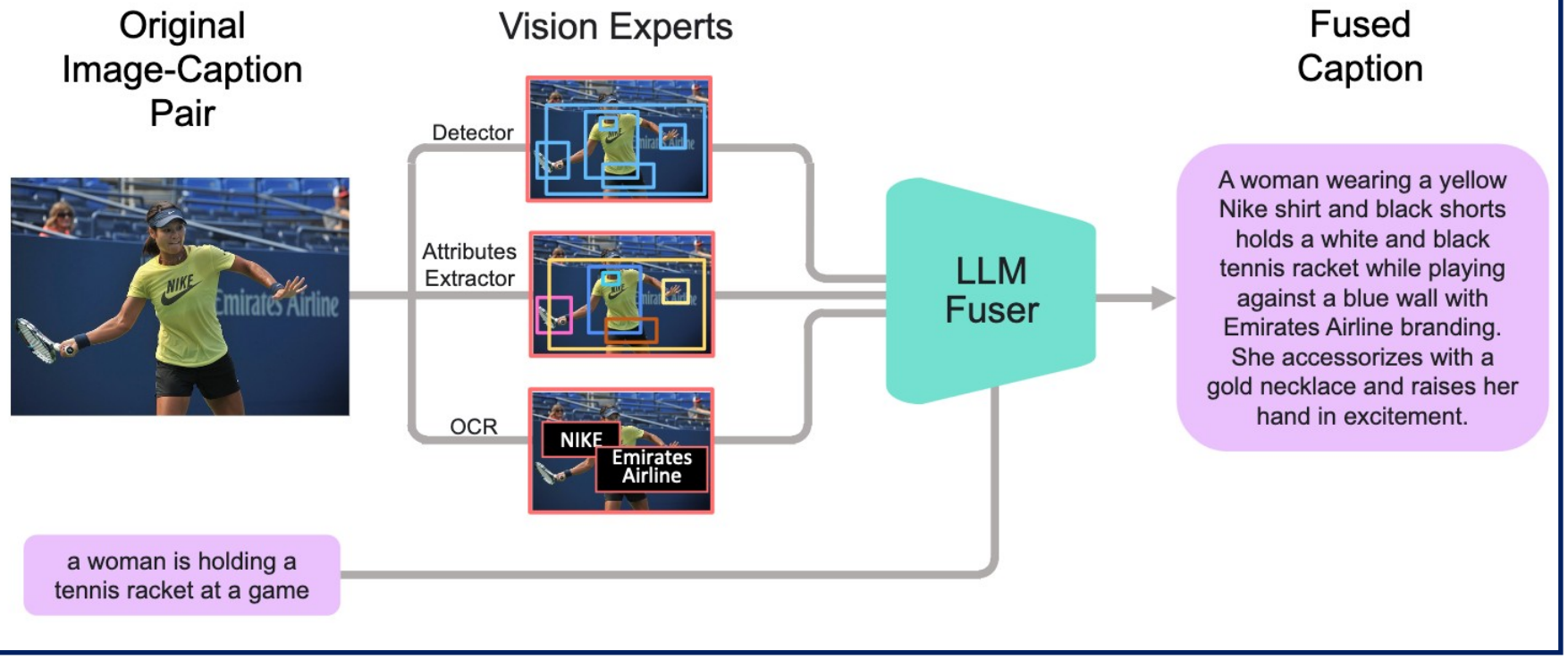


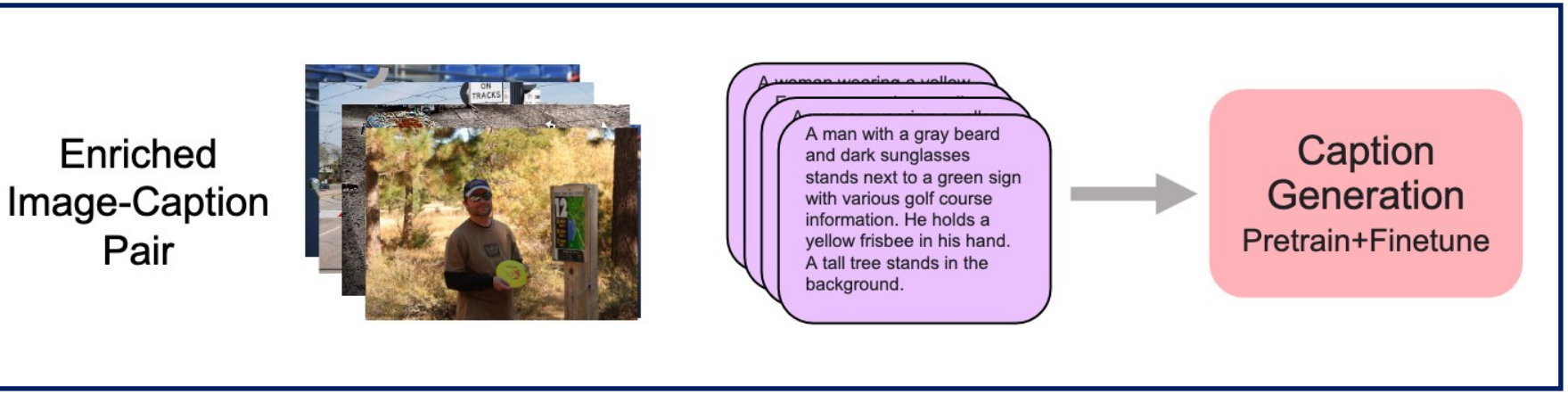
FuseCap Framework

Captions with Vision Experts into LLM -> Fused Caption

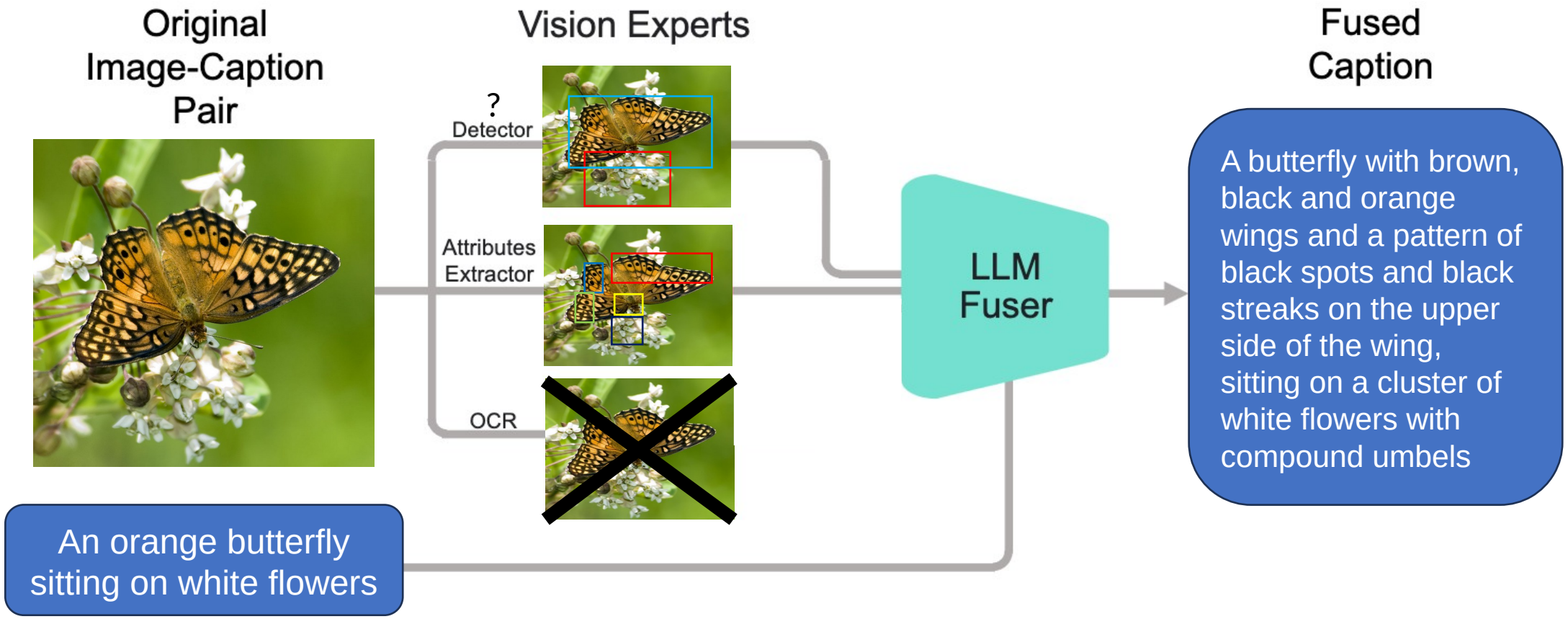
Fine-tune the Captioner based on Fused Captions

Module 1: Image Captioner





Module 1: Image Captioner

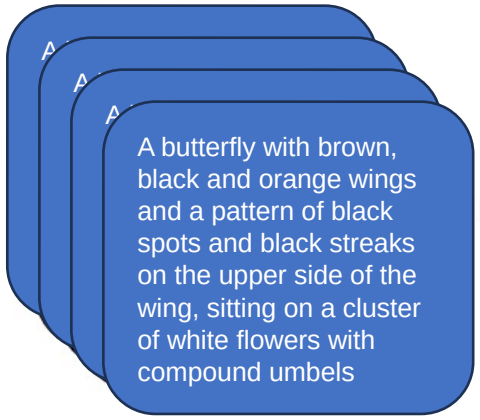


Object detection?
Biologically-relevant morphological attribute detectors?

Module 1: Image Captioner

Enriched Image-Caption Pair



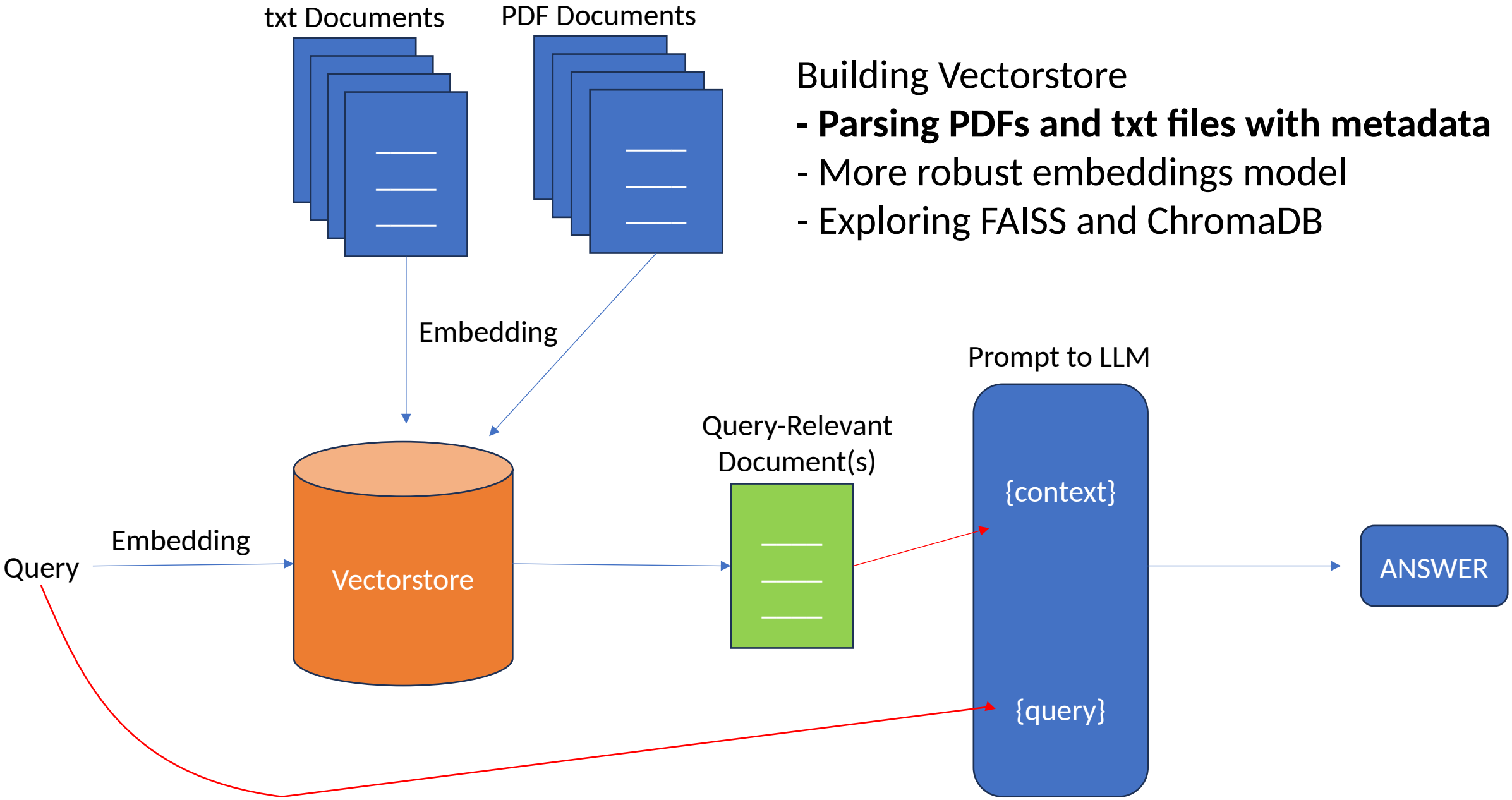


Caption Generation
Pretrain+Finetune

FuseCap finetuning with A100 GPUs x8

Image-taxonomy Dataset: Tree-of-Life 10M (iNat-21 backup)

Module 2: RAG Pipeline

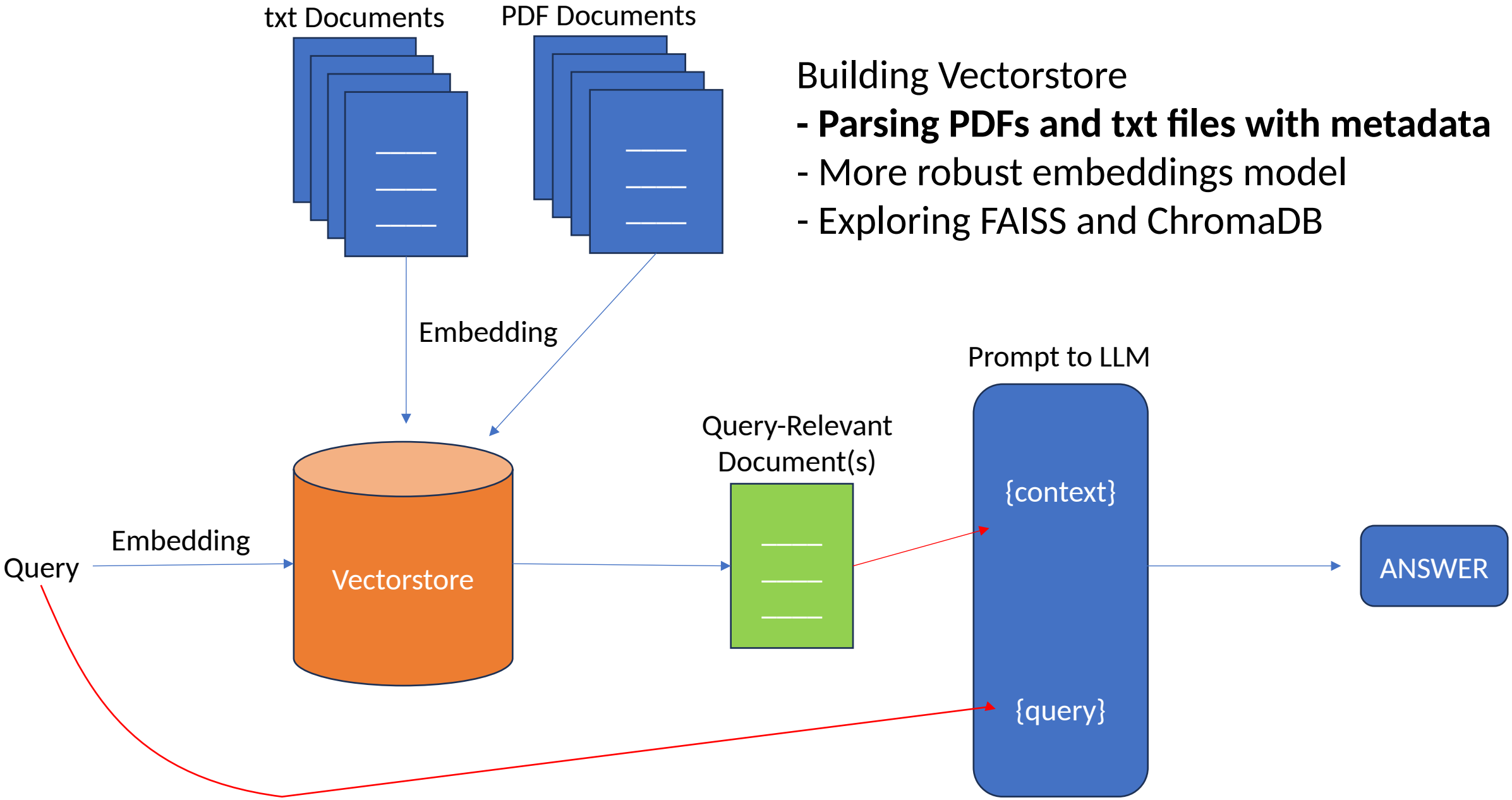


Building Vectorstore

- Parsing PDFs and txt files with metadata

- More robust embeddings model
- Exploring FAISS and ChromaDB

Module 2: RAG Pipeline

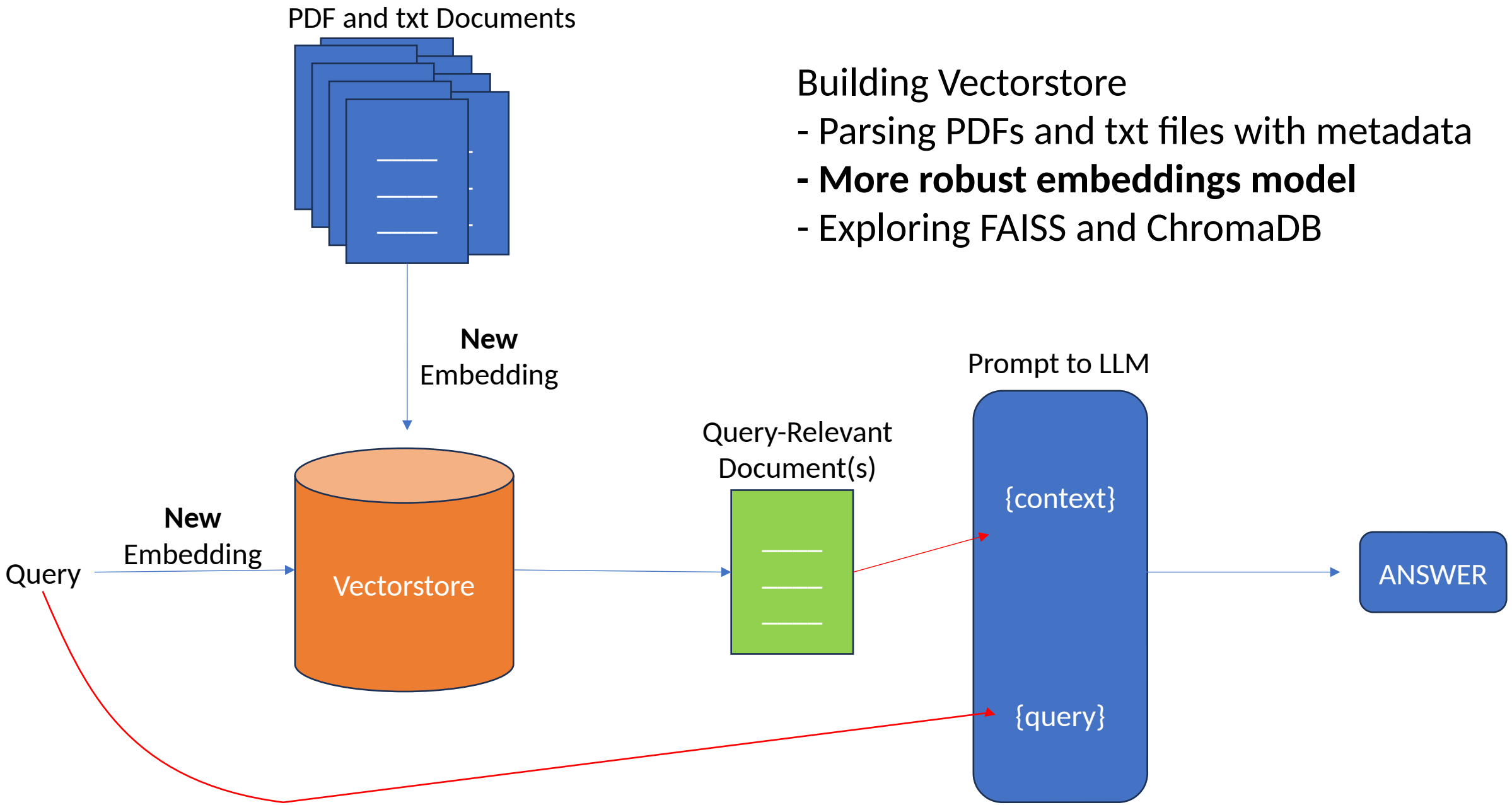


Building Vectorstore

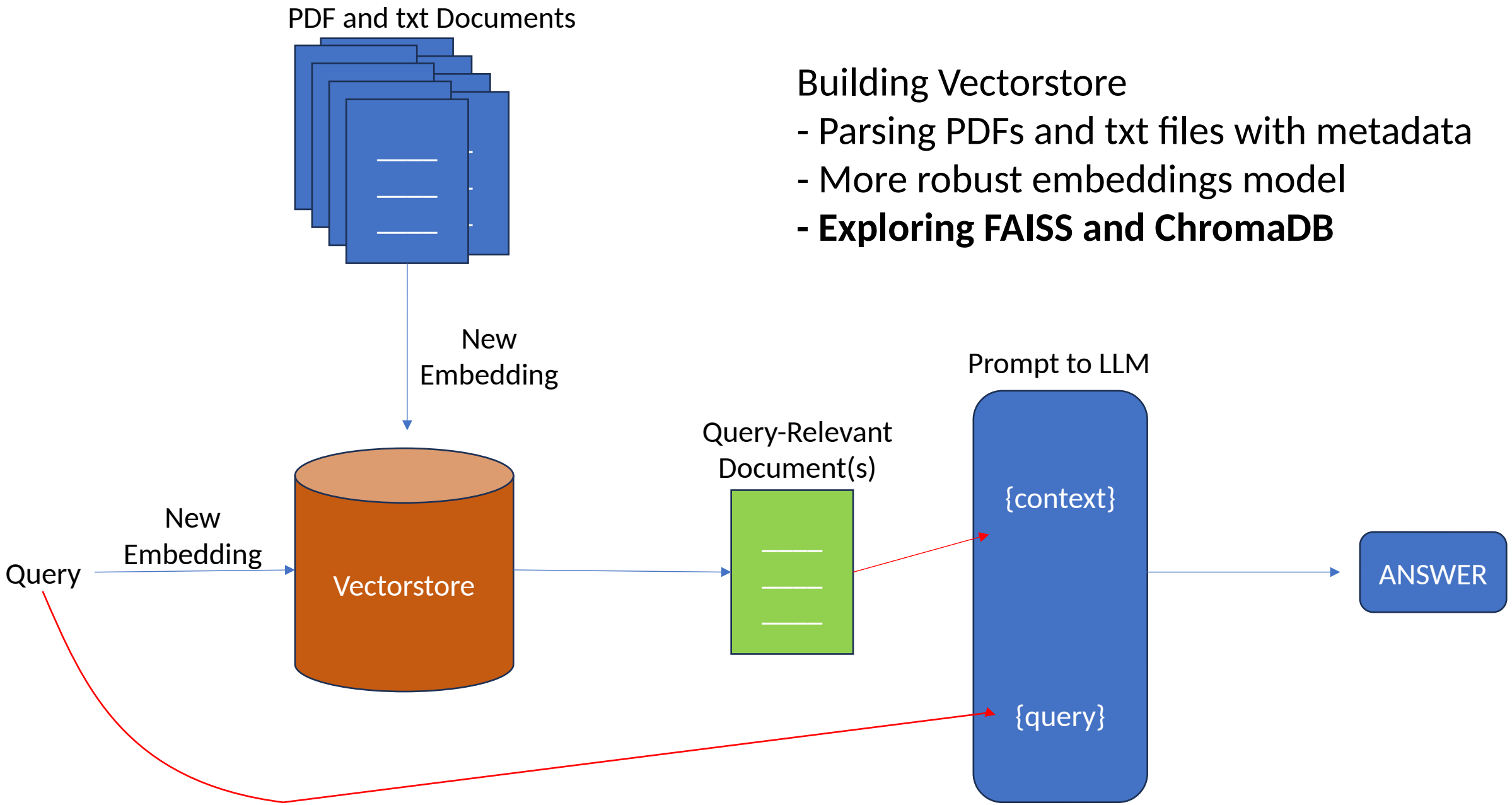
- Parsing PDFs and txt files with metadata

- More robust embeddings model
- Exploring FAISS and ChromaDB

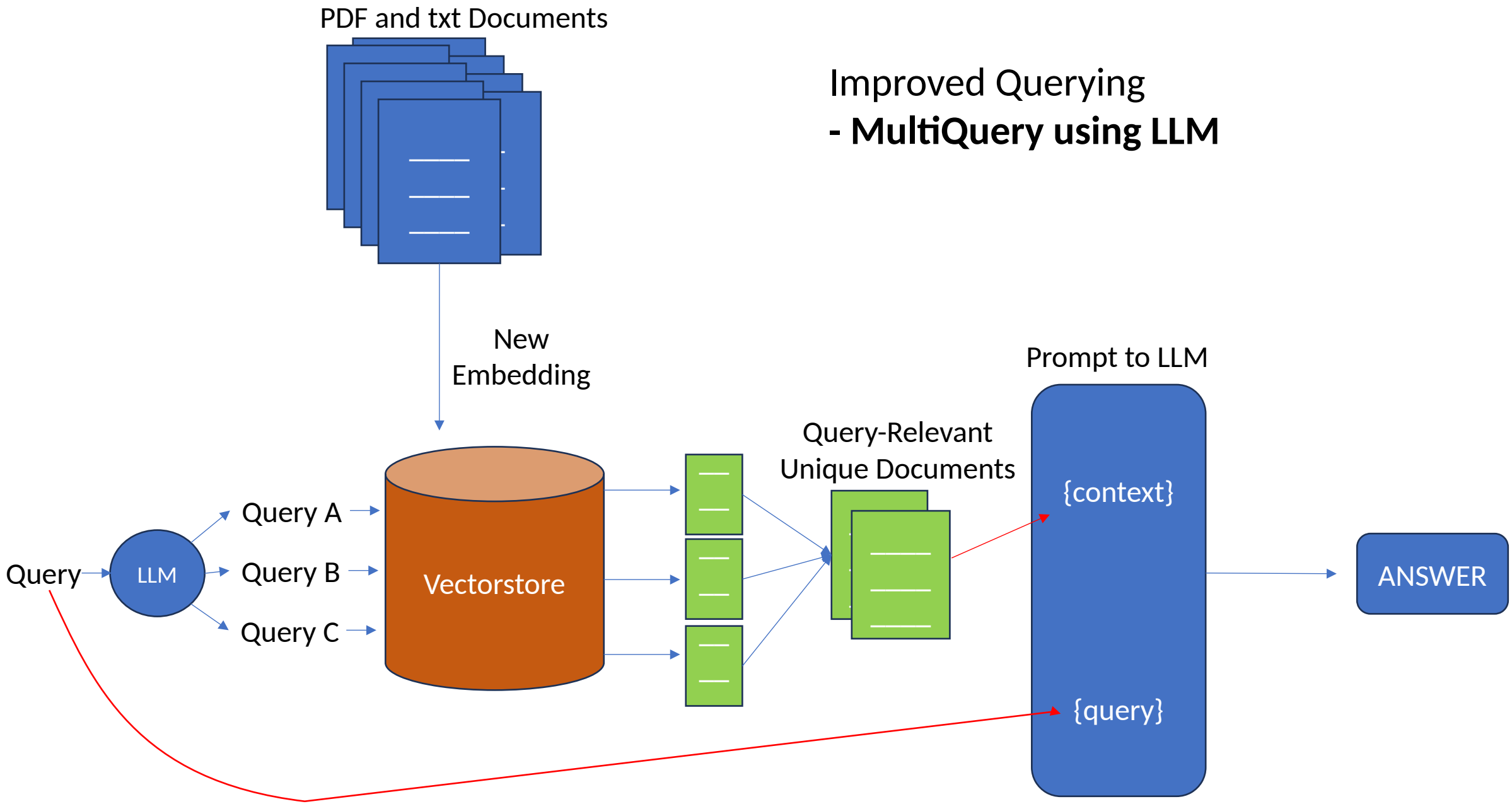
Module 2: RAG Pipeline



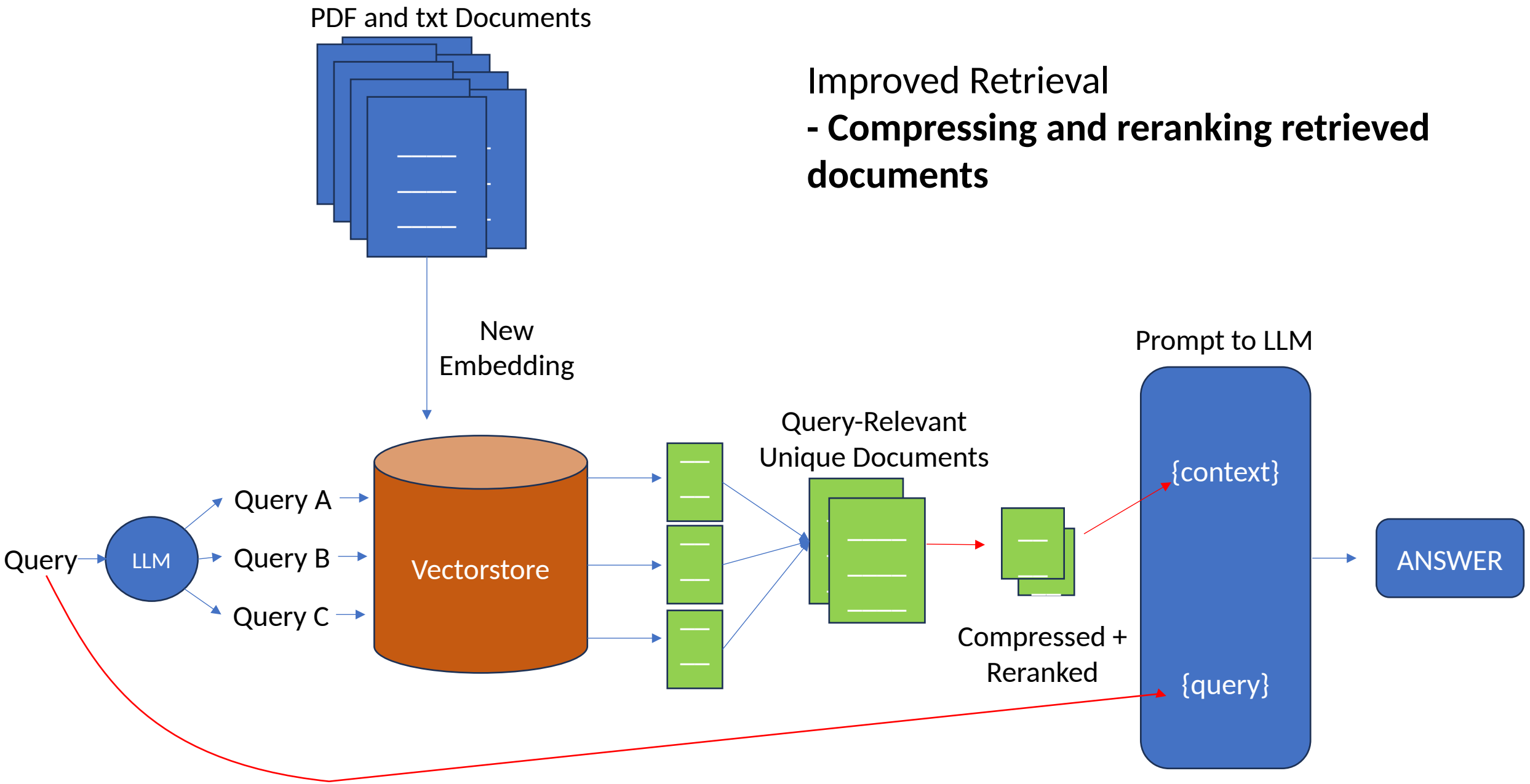
Module 2: RAG Pipeline



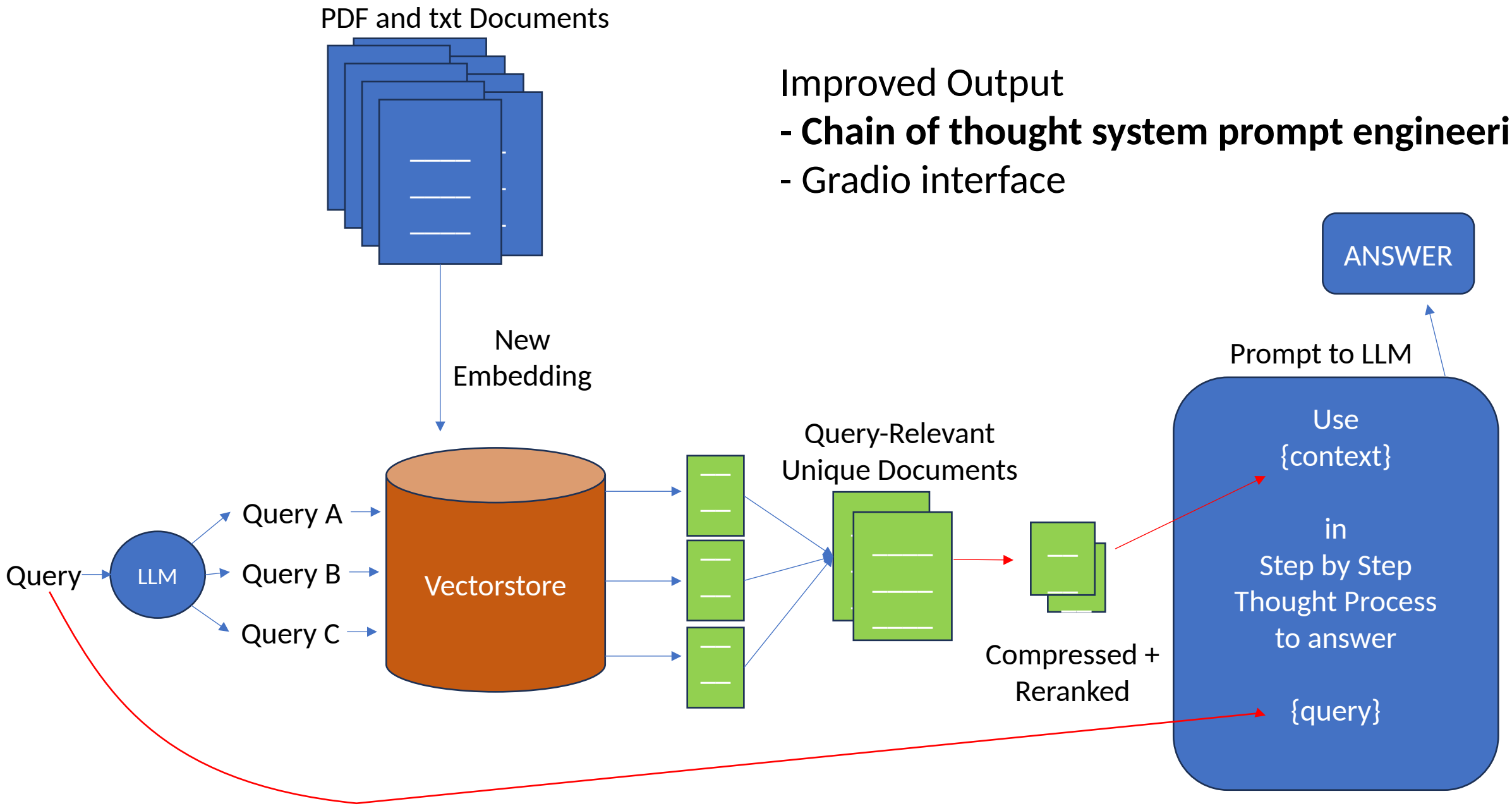
Module 2: RAG Pipeline



Module 2: RAG Pipeline



Module 2: RAG Pipeline

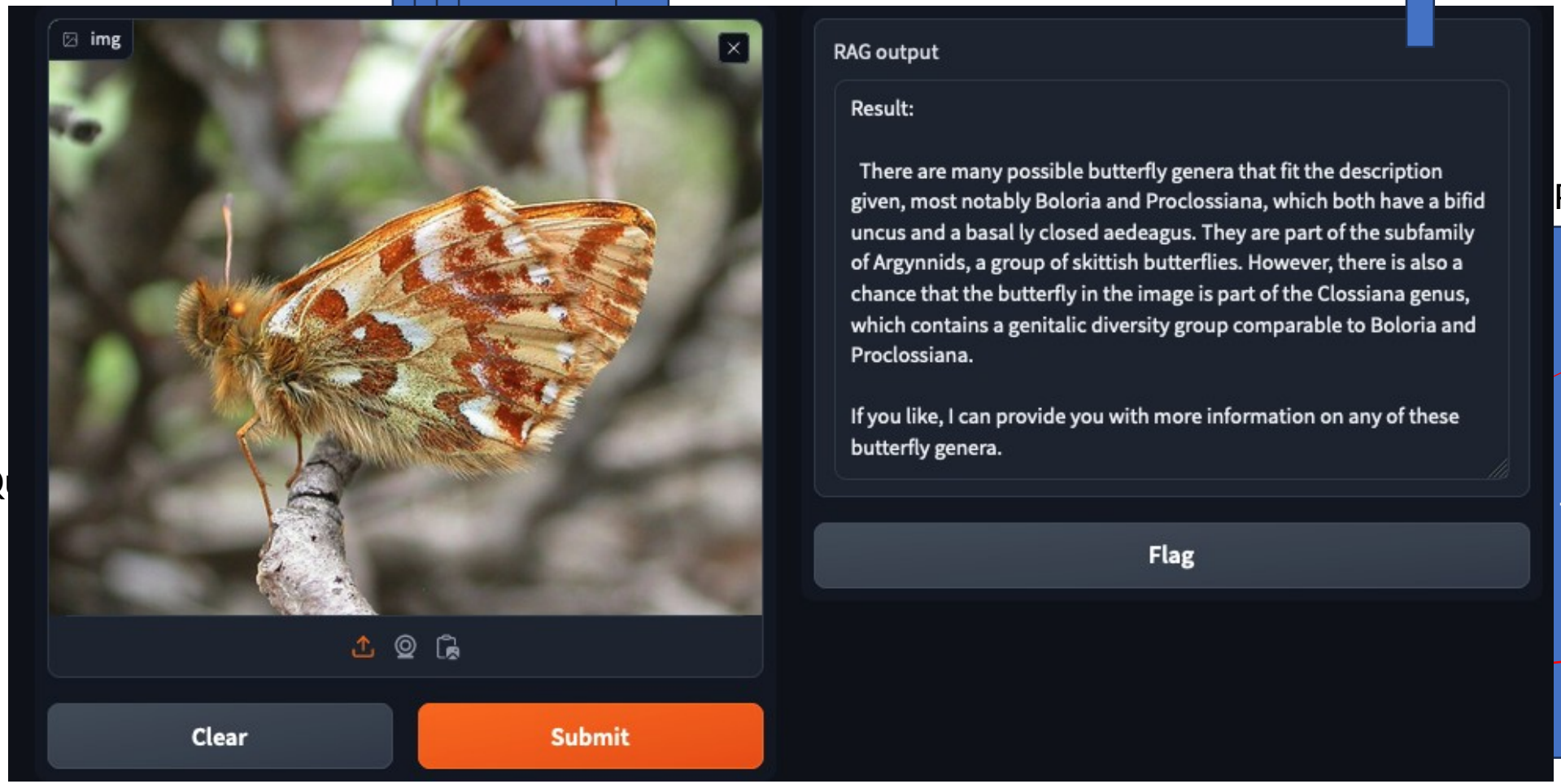


Module 2: RAG Pipeline

PDF and txt Documents



Improved Output
- Chain of thought system prompt engineering
- **Gradio interface**



ANSWER

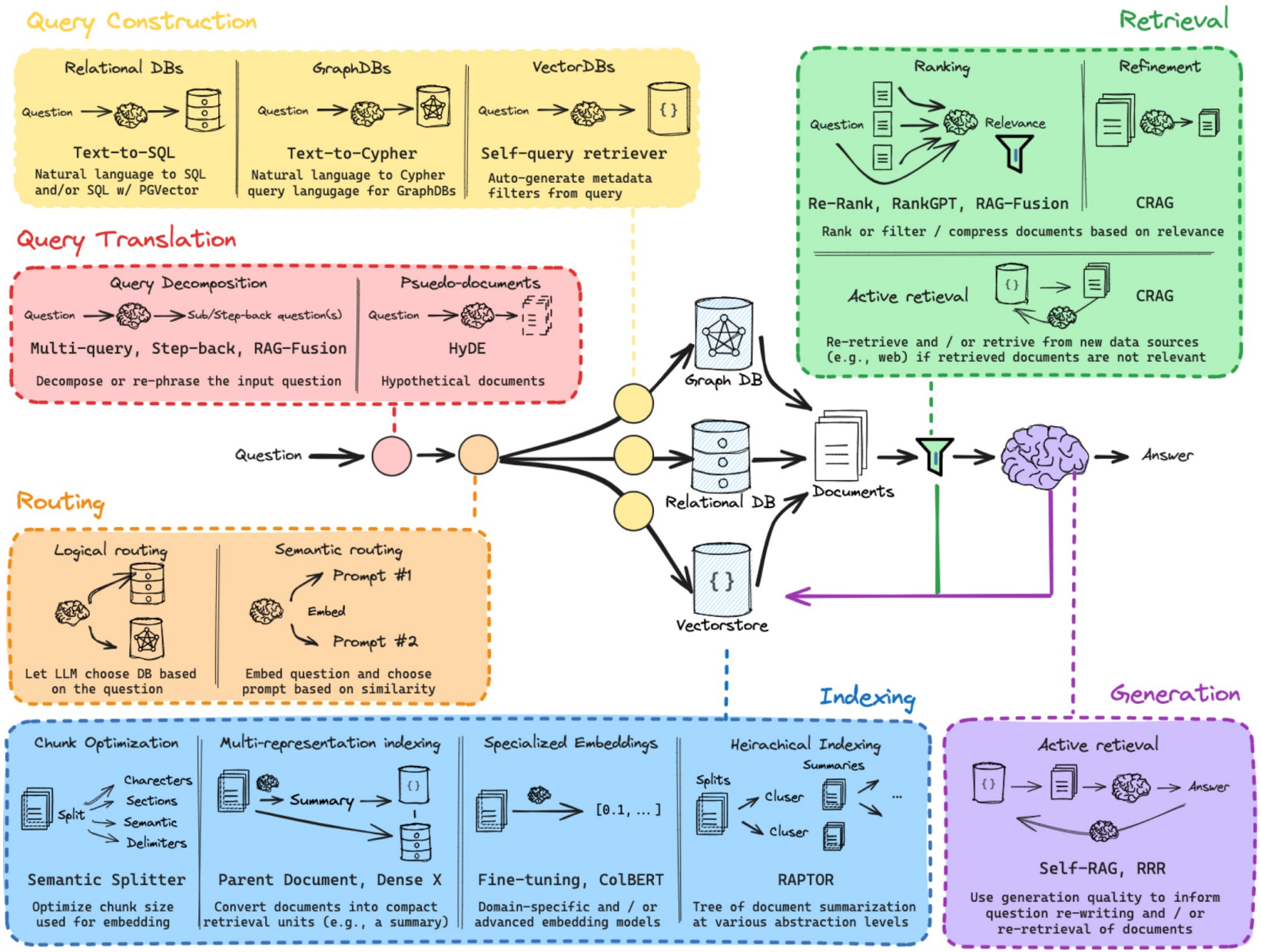
Prompt to LLM

Use
{context}

in
Step by Step
Thought Process
to answer

{query}

Possible RAG Extensions

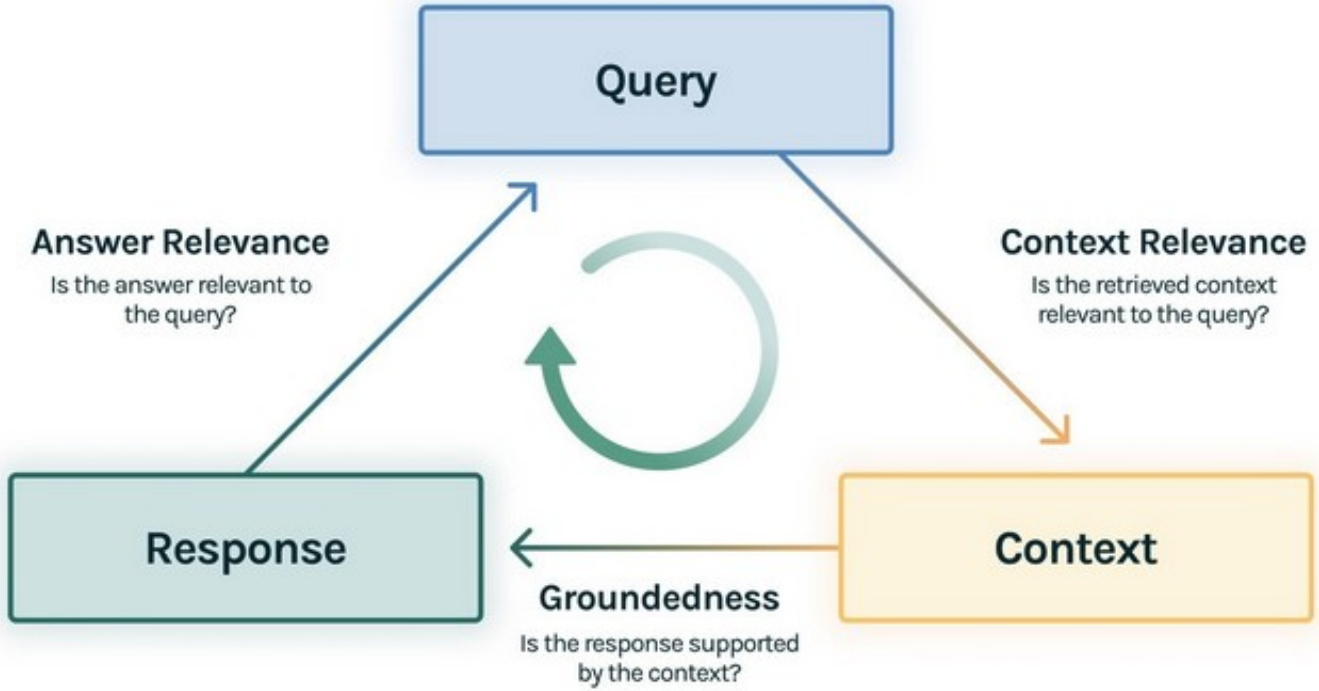


Lots of optimization options

Major Roadblocks:
- Web-scraping
- Evaluation

Next Steps (Sooner)

Immediate need for evaluation metrics (RAG Triad bv TruLens)



<u>Larger vectorstore:</u>

- Building multiple vectorstores for more directed querying
- Integrate metadata filters

Next Steps (Later)

MODULE 1: Image Captioner

Biologically-relevant fused captions

Pretraining and finetuning captioner

MODULE 2: RAG Pipeline

Integrating relational database (BOLD)?

Additional RAG optimizations

Full Pipeline:

Evaluation

- Taxonomic classification compared to benchmarks?
- How evaluate biodiversity-relevant knowledge?

Interactivity

- Chat integration?

Summary

Need for image-based classification paired with knowledge relevant to biodiversity

RAG is powerful tool to leverage LLMs and rich text-based biodiversity databases

Project still early in development but results-driven by nature of RAG and bootcamp

More research needed in prep for training image captioner

Evaluation metrics will help measure and guide improvement

Thank you ☺

Special thanks to Graham and Sujeevan for their hard work on this project and for including me!